

Sub-Agent Design

Hermes Agent Sub-Agent System Design

Table of Contents

- 1. System Overview
- 2. Core Architecture: Delegate Task
 - Overview
 - Key Design Constants
 - Two Execution Modes
- 3. Lifecycle and Isolation Mechanisms
 - Child Agent Construction
 - Context Isolation Strategy
 - Toolset Permission Tightening
- 4. Interrupt Propagation Mechanism
 - Three-Layer Interrupt Architecture
 - Interrupt Propagation Flow
 - Thread Safety of Interrupt Propagation
- 5. Credentials and Provider Resolution
 - Child Agent Credential Inheritance
 - Provider Resolution Priority
- 6. Progress Feedback and UI Integration
 - Dual-Path Progress Callbacks
 - `_last_resolved_tool_names` Global Pollution Issue
- 7. Iteration Budget
 - Budget Isolation Design
 - `execute_code` Refund Mechanism
- 8. Tool Call Truncation Protection
- 9. RL Environment Agent Loop
 - Comparison of Differences
 - Tool Scheduling Differences
- 10. ACP Session Manager
- 11. Mixture-of-Agents
- 12. Design Philosophy Summary
 - Core Principles
 - Differences from Traditional Multi-Agent
- 13. Notes and Known Pitfalls
 - Development Notes
 - Security Notes
 - Performance Notes
- 14. File Index

1. System Overview

The Sub-Agent system in Hermes Agent is not a "framework-style" multi-agent system, but rather a **tool-call-based task delegation mechanism**. Its core consists of the following modules:

Module	File Path	Responsibility
delegate_task tool	tools/delegate_tool.py	Sub-agent initiation, lifecycle management, concurrency control
AI Agent core class	run_agent.py	Sub-agent execution engine, conversation loop, interrupt propagation
Interrupt signal mechanism	tools/interrupt.py	Cross-thread interrupt notification (global threading.Event)
Auxiliary client routing	agent/auxiliary_client.py	Sub-agent Provider/Model resolution and credential management
Environment Agent Loop	environments/agent_loop.py	RL training environment dedicated agent loop
ACP Session Manager	acp_adapter/session.py	Multi-session management for VS Code and other editors
Mixture-of-Agents	tools/mixture_of_agents_tool.py	Multi-model aggregated reasoning tool (independent of delegation)

2. Core Architecture: Delegate Task

2.1 Overview

`delegate_task` is registered as an AI Agent subsystem **tool** (belonging to the **delegation** toolset). The main agent invokes this tool to **spawn subprocess-level sub-agents**, each with:

- Independent conversation history (isolated context)
- Independent terminal session (independent task_id)
- Restricted toolset (isolated permissions)
- Focused system prompt

Intermediate tool calls made by sub-agents **do not leak into the parent agent's context** — only a final summary is returned (i.e., context isolation).

2.2 Key Design Constants

```
# tools/delegate_tool.py
DELEGATE_BLOCKED_TOOLS = frozenset([
    "delegate_task", # Recursive delegation prohibited
    "clarify",       # User interaction prohibited
    "memory",        # Writing to shared MEMORY.md prohibited
    "send_message",  # Cross-platform side effects prohibited
    "execute_code",  # Sub-agents should reason step-by-step, not write scripts
])

MAX_CONCURRENT_CHILDREN = 3 # Max concurrent sub-agent count
MAX_DEPTH = 2               # Delegation depth limit (parent=0, child=1, grandchild=2 rejected)
DEFAULT_MAX_ITERATIONS = 50 # Default max iterations per sub-agent
DEFAULT_TOOLSETS = ["terminal", "file", "web"] # Default toolsets
```



Note

MAX_CONCURRENT_CHILDREN=3: A Cognitive Boundary, Not a Resource Limit

The origin of the number 3 is not a technical constraint, but a product intuition:

- Beyond 3 parallel actions, users cannot effectively track system behavior
- This shares the same roots as the "7±2 principle" — human cognitive capacity for parallel streams is extremely limited
- Needs that exceed 3 concurrent splits are essentially workflow design problems, and should not be solved by stacking delegation depth

Note

MAX_DEPTH=2: A Trust Chain, Not a Call Stack

The semantics of depth limitation is not about technical stack depth, but about **visibility boundaries**:

- Sub-agents introduce information loss (summary instead of details); after one more delegation layer, the parent cannot judge the quality of the grandchild's summary
- Error propagation paths in delegation chains become untraceable beyond two layers
- The essential constraint of MAX_DEPTH=2: delegation only makes sense when the parent can see the child's complete work

Note

Truncation Algorithm Design Principle: Concurrency Constraints Apply Only to Delegation Itself

- Concurrency limits constrain the "create sub-agent" action, not all tool calls
- Non-delegate tools in the same response (terminal, web_search, etc.) are unaffected by truncation
- Truncation and deduplication form a two-layer defense: truncate delegation first, then deduplicate results to prevent duplicate calls

2.3 Two Execution Modes

Single Task Mode:

```
delegate_task(goal="Fix bug #123", context="...", toolsets=["terminal", "file"])
```



Executes directly on the main thread with no thread pool overhead.

Batch Mode (Parallel):

```
delegate_task(tasks=[
    {"goal": "Task A", "context": "..."},
    {"goal": "Task B", "context": "..."},
    {"goal": "Task C", "context": "..."},
])
```



Executes concurrently via ThreadPoolExecutor, with up to MAX_CONCURRENT_CHILDREN=3 in parallel.

Note

Provider Chain: Capability Routing, Not Fallback Backup

Routing logic differs for different task types because the bottlenecks differ:

- Text tasks: bottleneck is "model selection space" → OpenRouter is optimal (hundreds of models)
- Vision tasks: bottleneck is "multimodal support completeness" → Main Provider is optimal (PDF, images, interleaved input)
- Vision tasks do not go through OpenRouter aggregation layer: third-party vision routing is unreliable; the main Provider's native support is more complete

Note

Batch Results: User-Perceived Orderly, Internal Concurrency Non-Blocking

- Concurrent execution, but return order matches input order (deterministic)
- Core algorithm: "First to complete, first to unpack, then return in original order" — task_index mapping decouples completion order from return order
- For single tasks, skip the thread pool and execute directly on the main thread to eliminate unnecessary overhead

3. Lifecycle and Isolation Mechanisms

3.1 Child Agent Construction (_build_child_agent)

Child agents are built synchronously on the **main thread** by the parent agent (thread-safe), then handed off to the thread pool:

```
# tools/delegate_tool.py:150-249
child = AIAgent(
    base_url=effective_base_url,
    api_key=effective_api_key,
    model=effective_model,
    provider=effective_provider,
    max_iterations=max_iterations,      # Independent iteration budget
    enabled_toolsets=child_toolsets,    # Restricted toolset (intersection with parent)
    quiet_mode=True,                   # Silent mode
    ephemeral_system_prompt=child_prompt,# Focused system prompt
    log_prefix=f"[subagent-{task_index}]",
    skip_context_files=True,           # Skip context files
    skip_memory=True,                  # Skip memory system
    session_db=parent_agent.session_db,# Shared session database
    tool_progress_callback=child_progress_cb, # Progress callback
    iteration_budget=None,              # Each sub-agent has independent budget
)
child._delegate_depth = parent._delegate_depth + 1
```



3.2 Context Isolation Strategy

Isolated (not inherited):

- Parent agent's conversation history (not inherited at all)
- Independently generated system prompt (Focused prompt)
- Independent task_id (independent terminal session state)
- Independent iteration budget (IterationBudget)

Inherited:

- API credentials (base_url, api_key, etc.)
- Parent agent's enabled_toolsets **intersection** (cannot exceed parent's capabilities)
- Parent agent's providers configuration

Blocked (forcibly removed):

- delegate_task (prevents recursion)
- clarify, memory, send_message, execute_code

3.3 Toolset Permission Tightening (Intersection Strategy)

```
# tools/delegate_tool.py:177-184
parent_toolsets = set(parent_agent.enabled_toolsets or DEFAULT_TOOLSETS)
if toolsets:
    child_toolsets = _strip_blocked_tools([t for t in toolsets if t in parent_toolsets])
elif parent_agent.enabled_toolsets:
    child_toolsets = _strip_blocked_tools(parent_agent.enabled_toolsets)
else:
    child_toolsets = _strip_blocked_tools(DEFAULT_TOOLSETS)
```

 A sub-agent can only obtain tools that the parent agent already has and that are not blocked. **Permission expansion is not allowed**, embodying the principle of least privilege.

Note

Toolset-Level Filtering vs. Tool-Name-Level Filtering: Namespace Convention Safety and Precise Blocking as Defense in Depth

There are two independent filtering mechanisms:

- Toolset level: filters entire permission collections by toolset name (e.g., `delegation`, `clarify`)
- Tool name level: precisely blocks by specific tool name (e.g., `delegate_task`, `clarify`)

Namespace convention filtering is simpler but fragile (if toolset names collide, blocking fails); precise blocking is more robust. The combination of both forms defense in depth.

Note

Intersection Empty Set: Silent Degradation, Not Fail-Fast

When the toolset requested by the user has an empty intersection with the parent agent's toolsets:

- No error is raised; instead, the sub-agent runs with an empty toolset
- The sub-agent fails implicitly during execution, returning **"failed"** status
- This is a security default of "give fewer tools rather than over-privilege tools"
- Trade-off: errors are not exposed at delegation time; configuration errors can only be found by digging into sub-agent logs

4. Interrupt Propagation Mechanism

4.1 Three-Layer Interrupt Architecture

Hermes Agent implements a three-layer interrupt propagation mechanism to ensure that when the user terminates the parent agent, all sub-agents immediately sense and exit.

Each layer has a clear division of responsibilities: Layer 1 handles cross-thread notification (any tool executing can sense an interrupt), Layer 2 controls agent main loop iteration decisions, and Layer 3 recursively propagates the interrupt signal to all sub-agents. Together, they ensure that the interrupt signal is responsive at both the fine-grained tool execution layer and the coarse-grained agent decision layer.

Layer 1 — Global Interrupt Event (`tools/interrupt.py`):

Process-level shared; all tools can query during execution. Used for early exit of time-consuming operations (code execution, external processes).

```
_interrupt_event = threading.Event() # Global singleton

def set_interrupt(active: bool): ...
def is_interrupted() -> bool: ...
```



Layer 2 — AI Agent Instance Interrupt Flag (`run_agent.py`):

Instance-level flag, read/written by the `interrupt()` method and at main loop boundaries. Used to decide whether to continue the next iteration.

```
class AI Agent:
    self._interrupt_requested = False
    self._interrupt_message = None
```



Layer 3 — Sub-Agent Linked List (`run_agent.py`):

Recursively propagates the interrupt signal from the parent agent to the collection of all sub-agent instances. Thread-safe.

```
self._active_children = [] # List of running child AI Agents
self._active_children_lock = threading.Lock()
```



4.2 Interrupt Propagation Flow

User inputs new message or Ctrl+C

↓

```
parent_agent.interrupt(message)
```

↓

1. Sets `self._interrupt_requested = True`
2. Sets global `_interrupt_event` (notifies all tools)
3. Iterates `_active_children`, recursively calls `interrupt()` on each sub-agent

↓

Sub-agent receives `interrupt()`

↓

Sub-agent.`run_conversation()` checkpoints detect `_interrupt_requested`:

- `_interruptible_api_call()`: checked in HTTP thread join loop
- `_interruptible_streaming_api_call()`: checked in streaming loop
- Throws `InterruptedError`, stops current iteration

↓

```
_run_single_child() catches exception, returns status="interrupted"
```



Note

Interrupt Semantics: Intent Switching, Not Operation Cancellation

When the user triggers an interrupt, the psychological expectation is "stop immediately," not "wait for completion":

- Partially accumulated tokens can be discarded (new intent doesn't need incomplete output)
- Fully accumulated tool call arguments must be preserved (this is safe input for subsequent execution)
- Fidelity priority: tool call arguments > streaming tokens
- The essence is understanding "interrupt" as intent switching, correctly choosing which state to preserve

Note

Three-Layer Interrupt: Layered Authorization of Perception Bus and Decision Signal

- Layer 1 (`_interrupt_event`): Tool-visible layer, passive perception — any tool can proactively query and exit early during execution
- Layer 2 (`_interrupt_requested`): Agent instance layer, proactive decision — checked at loop boundaries (before/after API calls), decides whether to continue iteration
- Layer 3 (`_active_children` linked list): Propagation layer — recursively notifies all sub-agents
- Two-layer signal combination: can be sensed during tool execution and responded to at agent decision points

4.3 Thread Safety of Interrupt Propagation

- `_active_children_lock` is a `threading.Lock`, protecting concurrent access to the sub-agent linked list
- Sub-agents remove themselves from `_active_children` in the `finally` block of `_run_single_child`
- The `interrupted` status is correctly reflected in the delegation tool's JSON return value

Note

Streaming Interrupt Latency: At Most One Chunk, Not One Request

- Interrupt checks are inserted into the streaming loop, rather than using blocking thread joins — no additional waiting is introduced
- Response latency upper bound: $\min(\text{next chunk arrival time}, \text{STREAM_READ_TIMEOUT})$
- This is a pragmatic trade-off between interrupt timeliness and streaming throughput

5. Credentials and Provider Resolution

5.1 Child Agent Credential Inheritance

`delegate_task` supports configuring **independent Provider/Model** for sub-agents, rather than simply inheriting the parent agent's settings:

```
# tools/delegate_tool.py:570-656
def _resolve_delegation_credentials(cfg, parent_agent) -> dict:
    # Three configuration approaches:
    # 1. delegation.base_url → Direct OpenAI-compatible endpoint
    # 2. delegation.provider → Resolved through runtime_provider system
    # 3. No configuration → Fully inherit parent agent's credentials
```



Supports routing sub-agents to different Provider:Model combinations via `delegation.provider`, for example:

- Parent agent runs on Nous Portal (strong reasoning model)
- Sub-agent can be routed to OpenRouter (fast, cheap model)

5.2 Provider Resolution Priority

`agent/auxiliary_client.py` defines the complete auxiliary task Provider chain:

Text task auto-detection order:

1. OpenRouter (OPENROUTER_API_KEY)
2. Nous Portal (OAuth, `~/hermes/auth.json`)
3. Custom endpoint (`config.yaml` + OPENAI_API_KEY)
4. Codex OAuth (Responses API)
5. Native Anthropic
6. Direct API Key providers (z.ai/GLM, Kimi, MiniMax)

Vision task order:

1. Main Provider (if known vision backend)
2. OpenRouter → Nous → Codex → Anthropic → Custom

6. Progress Feedback and UI Integration

6.1 Dual-Path Progress Callbacks

```
# tools/delegate_tool.py:78-147
def _build_child_progress_callback(task_index, parent_agent, task_count) -> callable:
    # Returns a callback(tool_name, preview=None)

    # Path 1: CLI (KawaiiSpinner tree view)
    if spinner:
        spinner.print_above(f" {prefix}|— 🌀 {tool_name}: {preview}")

    # Path 2: Gateway (batch tool name aggregation)
    if parent_cb:
        _batch.append(tool_name)
        if len(_batch) >= 5: # Batch size
            parent_cb("subagent_progress", f"🌀 {summary}")
        # Flush remaining on completion
```



- CLI mode: prints tree-structured progress above the spinner via `KawaiiSpinner.print_above()`
- Gateway mode: batch-forwards to the messaging platform via `tool_progress_callback`
- Tool callbacks (`tool_progress_callback`) use batching to avoid message flooding

Note

Sub-agents Do Not Inherit Conversation History: Transfer Document, Not Continuation of Memory

Core assumption: subtask == does not need the parent task's process memory, only goals and background constraints ==

- Analogy to human work: transfer documents do not record processes, only conclusions and constraints
- At delegation time, the parent agent is forced to perform "cognitive compression" — if you can't write a concise description, the task boundary itself is unclear
- Side effect: long-horizon reasoning tasks (requiring sustained exploration paths) may suffer duplicate exploration due to isolation

Note

Dual-Path Progress Feedback: Different Platforms, Different Attention Models

- CLI users: developers, focused waiting, ample terminal space, need visible reasoning process (`_thinking` summary visible, tree indentation reasonable)
- Gateway users: async notifications, fragmented attention, don't need reasoning logs, only need to know "it's working"
- batch_size=5: trade-off between real-time feedback and message channel pressure (single send is best, but some platforms have rate limits)

Note

`_thinking` Events: Visible in CLI, Deliberately Filtered in Gateway

- `_thinking` content shows first 55 characters in CLI display, helping users understand the reasoning process

- Gateway (Slack, Feishu, etc.) does not relay `_thinking`: reasoning logs form noisy messages; the audience doesn't need streaming reasoning details
- `_flush()` mechanism: called only in the Gateway path, ensuring that the last few events below `batch_size` are also sent when sub-agents exit

6.2 `_last_resolved_tool_names` Global Pollution Issue

Problem Description:

`model_tools._last_resolved_tool_names` is a **process-level global variable** that gets overwritten by the sub-agent's toolset when `_build_child_agent()` calls `AIAgent.__init__() → get_tool_definitions()`, causing subsequent parent agent tools like `execute_code` to receive incorrect tool lists.

Solution (implemented in `delegate_tool.py`):

```
# tools/delegate_tool.py:471-496
# Save parent tool names before building any sub-agent
_parent_tool_names = list(model_tools._last_resolved_tool_names)

try:
    for i, t in enumerate(task_list):
        child = _build_child_agent(...)
        child._delegate_saved_tool_names = _parent_tool_names # Mount on child object
        children.append((i, t, child))
finally:
    # Even if sub-agent construction fails, restore global state
    model_tools._last_resolved_tool_names = _parent_tool_names

# In _run_single_child's finally block, restore
finally:
    saved_tool_names = getattr(child, "_delegate_saved_tool_names", None)
    if isinstance(saved_tool_names, list):
        model_tools._last_resolved_tool_names = list(saved_tool_names)
```



7. Iteration Budget

7.1 Budget Isolation Design

```
# run_agent.py:170-180
class IterationBudget:
    """Thread-safe iteration counter.

    Parent agent's budget ceiling is max_iterations (default 90).
    Each sub-agent has an independent budget ceiling delegation.max_iterations (default 50).
    Total iterations across parent + sub-agents can exceed the parent's max_iterations.
    """
```



Key Design Decision:

- Sub-agents **do not share** the parent's iteration budget (`iteration_budget=None` → creates a new independent budget)
- This is designed to prevent sub-agents from consuming the parent's iteration budget, leaving the parent unable to complete wrap-up work
- Users can configure the sub-agent budget ceiling via `delegation.max_iterations`

Note

Iteration Budget Isolation: Capability Boundary Design, Not Billing Protection

- Independent budget = maximum capability release, not cost protection (total iterations can far exceed the parent's `max_iterations`)
- Risk of shared budget: sub-agents may exhaust the parent's budget, leaving the parent unable to complete wrap-up work
- `execute_code` refund logic: pure script-writing rounds do not consume reasoning budget — tokens are reserved for steps that genuinely need LLM reasoning

7.2 `execute_code` Refund Mechanism

```
# run_agent.py:7746-7748
_tc_names = {tc.function.name for tc in assistant_message.tool_calls}
if _tc_names == {"execute_code"}:
    self.iteration_budget.refund() # Refund, not counted toward iteration limit
```



When an iteration round only calls `execute_code` (programmatic tool call), that iteration does not consume the budget, preventing script-writing from exhausting the iteration count.

Note

`execute_code` Refund: Precise Equality, Not Subset Check

- Precise set equality: only refund when the **only** tool call in this round is `execute_code`
- `execute_code` + any other tool → No refund (considered "script-assisted operation," charged normally)
- This is a white-box trust design: pure script-writing rounds don't need to consume reasoning budget

8. Tool Call Truncation Protection

```
# run_agent.py:2807-2835
@staticmethod
def _cap_delegate_task_calls(tool_calls: list) -> list:
    """Truncates excess delegate_task calls to MAX_CONCURRENT_CHILDREN.

    Prevents the model from calling delegate_task multiple times in one response,
    bypassing the MAX_CONCURRENT_CHILDREN=3 concurrency limit.
    """
```



Even if the model generates multiple `delegate_task` calls in a single LLM response, the system automatically truncates to at most 3, preventing resource exhaustion attacks.

9. RL Environment Agent Loop

`HermesAgentLoop` in `environments/agent_loop.py` is a dedicated agent loop for **reinforcement learning training environments**. It shares tool scheduling logic with the main agent (`run_agent.py`) but has independent design:

9.1 Comparison of Differences

Feature	<code>run_agent.py</code> <code>AIAgent</code>	<code>HermesAgentLoop</code>
Execution mode	Synchronous (main thread blocking)	Asynchronous (<code>async def run()</code>)
Tool execution	Synchronous <code>handle_function_call()</code>	<code>loop.run_in_executor()</code> thread pool

Feature	run_agent.py AIAgent	HermesAgentLoop
Thread pool size	Main thread	Configurable (resize_tool_pool(), default 128)
State persistence	SessionDB	ManagedServer (RL environment)
Interrupt mechanism	Thread interrupt flag	None (not applicable in RL environment)
ToDoStore	Global	Ephemeral, created fresh each round

9.2 Tool Scheduling Differences

```
# environments/agent_loop.py:394-407
# Todo/memory/session_search handled specially inside HermesAgentLoop
if tool_name == "todo":
    tool_result = _todo_tool(todos=args.get("todos"), store=_todo_store)
elif tool_name == "memory":
    tool_result = '{"error": "Memory is not available in RL environments."}'
else:
    loop = asyncio.get_event_loop()
    tool_result = await loop.run_in_executor(
        _tool_executor, # 128-worker thread pool
        lambda: handle_function_call(tool_name, args, task_id=task_id),
    )
```



10. ACP Session Manager (Multi-Editor Integration)

SessionManager in acp_adapter/session.py provides multi-session management for IDE integration scenarios:

- Each VS Code / Zed / JetBrains session corresponds to an independent SessionState
- Each session contains one AIAgent instance
- Supports session fork (deep copy history to new session)
- Supports session persistence (resume after restart)
- Each session has an independent cancel_event (thread-level cancellation)

11. Mixture-of-Agents (Independent of Delegation)

tools/mixture_of_agents_tool.py implements a MoA architecture based on research paper [arXiv:2406.04692v1](#), **completely independent of delegate_task**:

Two-Layer Architecture:

Layer 1 (Reference models, parallel):

- claude-opus-4.6 / gemini-3-pro / gpt-5.4-pro / deepseek-v3.2

Layer 2 (Aggregation model):

- claude-opus-4.6 → Synthesizes final answer



- Each reference model is called in parallel via OpenRouter API
- Failure tolerance: MIN_SUCCESSFUL_REFERENCES=1 (only 1 success required)
- Retry mechanism: exponential backoff (up to 6 times)
- Only suitable for extremely complex tasks requiring multi-perspective reasoning

Note

MoA vs. Delegation: Quality Enhancement vs. Capability Expansion

The key difference lies in the **direction of information flow**:

- Delegation: divergence-convergence (parent → multiple children → aggregation), suitable for "doing more things"
- MoA: parallel (multiple independent sources → single aggregator), suitable for "doing one thing more correctly"

The two are complementary — future direction: first use MoA to generate candidate plans for subtasks, then use Delegation to distribute to specialized agents for execution

Note

Key Parameter Design in MoA

- Relaxed gate (MIN_SUCCESSFUL_REFERENCES=1): ≥1 reference model success is sufficient to proceed, allowing 3 out of 4 to fail
- Fault tolerance internalization: each model internally catches exceptions and returns False; **gather** won't crash due to single model failure
- Independent exponential backoff: each model's retries proceed independently; worst-case wait time = max(individual model times), not sum
- Temperature asymmetry: reference layer 0.6 (high diversity) → aggregation layer 0.4 (low temperature, faithful integration)

Note

System Mainline: Controlled Openness

Three trust stopping points, forming progressive decentralization:

- Parent to child: trust stops at summary (don't trust intermediate process); child cannot delegate
- Child to itself: trust stops at iteration budget (don't trust unlimited execution); cannot clarify
- System to model: trust stops at tool truncation (don't trust anomalous responses)

This contradicts the industry trend of "giving agents as much capability as possible" — Hermes Agent's choice is **the stronger the capability, the tighter the constraints**, which is more defensive from an AI Safety perspective.

12. Design Philosophy Summary

12.1 Core Principles

1. **Least Privilege Isolation**: Sub-agents cannot obtain tools the parent agent lacks; blocked tools are forcibly removed
2. **Context Purity**: Sub-agent intermediate results do not leak into parent context, preventing context pollution
3. **Resource Bounded**: Depth limit (MAX_DEPTH=2), concurrency limit (MAX_CONCURRENT=3), iteration limit
4. **Thread Safety**: Cross-thread collaboration via locks, global state save/restore, and interrupt events
5. **Flexible Routing**: Sub-agents can independently choose different Provider/Model from the parent
6. **Graceful Degradation**: Provider automatic fallback, partial results returned on batch interruption

12.2 Differences from Traditional Multi-Agent

Dimension	Traditional Multi-Agent System	Hermes Sub-Agent
Communication protocol	Direct inter-agent message passing	Via tool calls (tool_call)
State sharing	Shared memory/blackboard	Complete isolation, independent budgets
Hierarchy	Flat or hierarchical	Tree hierarchy (MAX_DEPTH=2)

Dimension	Traditional Multi-Agent System	Hermes Sub-Agent
Context	Mutually readable	Parent only sees summary
Applicable scenarios	Open collaboration, debate	Task decomposition, independent execution

13. Notes and Known Pitfalls

13.1 Development Notes

- 1. `_last_resolved_tool_names` Global Pollution**
 - Resolved in `delegate_tool.py` via save/restore mechanism, but any new process-level global variable could cause similar issues
 - New code that depends on this global must consider the overwrite risk during sub-agent construction
- 2. Implicit Behavior of Depth Limits**
 - Parent agent's `_delegate_depth=0`, `child=1`, `grandchild=2` is rejected
 - When the parent agent's task triggers `delegate_task` twice (first delegation, then child delegates again), the second attempt silently fails
- 3. Batch Return Order**
 - Results are returned sorted by `task_index`, ensuring consistency with input order (`results.sort(key=lambda r: r["task_index"])`)
- 4. Sub-agent `iteration_budget` Independence**
 - Parent agent's 90 iterations and sub-agent's 50 iterations are **counted independently**
 - Total iterations can far exceed 90; users should monitor cost control
- 5. Gateway Progress Callback Batch Delay**
 - Gateway progress callbacks send only every 5 tool names
 - The final remaining less than 5 are sent via `_flush()` after sub-agent completion
- 6. `execute_code` Budget Refund**
 - Only refunded when the **only** tool call in this round is `execute_code`
 - If `execute_code` and other tools are mixed in one round, no refund
- 7. ACP Session CWD Override**
 - ACP adapter registers separate CWD for each task `id` via `register_task_env_overrides()`
 - Different editor sessions share the same `~/.hermes/state.db`, isolated by `session_id`

13.2 Security Notes

- 1. `clarify` is prohibited in Sub-agents:** Blocked tools list includes `clarify`; sub-agents cannot request clarification from users
- 2. Writing to shared memory is prohibited:** Sub-agents cannot call the `memory` tool, preventing race conditions from corrupting `MEMORY.md`
- 3. Toolset permissions are not expanded:** Sub-agent's toolset is the intersection of parent toolset and requested configuration

13.3 Performance Notes

- 1. Sub-agent construction is on the main thread:** `_build_child_agent()` executes on the main thread; sub-agent's `get_tool_definitions()` time counts toward the main thread
- 2. Thread pool size:** `HermesAgentLoop`'s tool thread pool defaults to 128, suitable for 89+ concurrent RL tasks
- 3. Credential resolution overhead:** `_resolve_delegation_credentials()` re-resolves `Provider` on each delegation, which may have I/O overhead

14. File Index

```
hermes-agent/
├── tools/
│   ├── delegate_tool.py      ← Sub-agent delegation core logic
│   ├── interrupt.py          ← Global interrupt signal
│   └── mixture_of_agents_tool.py ← MoA independent tool
├── agent/
│   ├── auxiliary_client.py   ← Provider/Model resolution routing
│   └── display.py            ← KawaiiSpinner UI
├── run_agent.py              ← AIAgent main class (includes interrupt propagation)
├── environments/
│   └── agent_loop.py         ← RL environment dedicated async Agent Loop
├── acp_adapter/
│   ├── session.py           ← ACP session management
│   └── entry.py              ← ACP entry point
├── batch_runner.py           ← Batch AIAgent parallel execution
└── rl_cli.py                 ← RL training CLI
```

